

# SQL Interview Questions & Answers

## Sales Trend Analysis & Aggregations

### Task 6: Data Analyst Internship - Elevate Labs

#### Interview Questions with Detailed Answers

##### Question 1: How do you group data by month and year?

**Answer:**

To group data by month and year in SQL, you use the `EXTRACT()` function combined with `GROUP BY` clause.

**SQL Example:**

```
SELECT
    EXTRACT(YEAR FROM Date) as Year,
    EXTRACT(MONTH FROM Date) as Month,
    SUM(Total_Revenue) as Monthly_Revenue
FROM sales_data
GROUP BY EXTRACT(YEAR FROM Date), EXTRACT(MONTH FROM Date)
ORDER BY Year, Month;
```

**Key Points:**

- `EXTRACT(YEAR FROM Date)` extracts the year component
- `EXTRACT(MONTH FROM Date)` extracts the month component
- `GROUP BY` groups rows with the same year and month
- Always include extracted fields in `GROUP BY` clause

**Alternative Methods:**

- PostgreSQL: `DATE_TRUNC('month', Date)`
- MySQL: `YEAR(Date)` and `MONTH(Date)`
- SQL Server: `DATEPART(YEAR, Date)` and `DATEPART(MONTH, Date)`

##### Question 2: What's the difference between `COUNT(*)` and `COUNT(DISTINCT col)`?

**Answer:**

| Aspect         | <code>COUNT(*)</code> | <code>COUNT(DISTINCT col)</code> |
|----------------|-----------------------|----------------------------------|
| <b>Purpose</b> | Counts all rows       | Counts unique values only        |

| Aspect               | COUNT(*)            | COUNT(DISTINCT col)       |
|----------------------|---------------------|---------------------------|
| <b>NULL Handling</b> | Includes NULL rows  | Ignores NULL values       |
| <b>Duplicates</b>    | Includes duplicates | Removes duplicates        |
| <b>Performance</b>   | Faster              | Slower (requires sorting) |

### SQL Examples:

```
-- COUNT(*) - counts all rows including duplicates
SELECT COUNT(*) as Total_Rows
FROM sales_data;

-- COUNT(DISTINCT) - counts unique values only
SELECT COUNT(DISTINCT Transaction_ID) as Unique_Transactions
FROM sales_data;

-- Practical comparison
SELECT
    COUNT(*) as Total_Orders,
    COUNT(DISTINCT Transaction_ID) as Unique_Transactions,
    COUNT(DISTINCT Product_Category) as Unique_Categories
FROM sales_data;
```

### When to Use:

- Use `COUNT(*)` for total record count
- Use `COUNT(DISTINCT col)` for unique value analysis
- Use `COUNT(col)` to exclude NULLs but include duplicates

## Question 3: How do you calculate monthly revenue?

### Answer:

Monthly revenue calculation involves grouping data by month and summing the revenue column.

### Basic SQL Query:

```
SELECT
    EXTRACT(YEAR FROM Date) as Year,
    EXTRACT(MONTH FROM Date) as Month,
    SUM(Total_Revenue) as Monthly_Revenue
FROM sales_data
GROUP BY EXTRACT(YEAR FROM Date), EXTRACT(MONTH FROM Date)
ORDER BY Year, Month;
```

### Enhanced Analysis:

```
SELECT
    EXTRACT(YEAR FROM Date) as Year,
    EXTRACT(MONTH FROM Date) as Month,
    SUM(Total_Revenue) as Monthly_Revenue,
```

```

COUNT(DISTINCT Transaction_ID) as Order_Count,
ROUND(AVG(Total_Revenue), 2) as Avg_Order_Value,
MIN(Total_Revenue) as Min_Order,
MAX(Total_Revenue) as Max_Order
FROM sales_data
GROUP BY EXTRACT(YEAR FROM Date), EXTRACT(MONTH FROM Date)
ORDER BY Year, Month;

```

#### Best Practices:

- Always include year to avoid mixing data across years
- Use `ROUND()` for currency formatting
- Consider time zones for accurate monthly boundaries
- Handle NULL values appropriately

### Question 4: What are aggregate functions in SQL?

#### Answer:

Aggregate functions perform calculations on multiple rows and return a single result value.

#### Common Aggregate Functions:

| Function                | Purpose            | Example                            |
|-------------------------|--------------------|------------------------------------|
| <code>SUM()</code>      | Total of values    | <code>SUM(Total_Revenue)</code>    |
| <code>COUNT()</code>    | Number of rows     | <code>COUNT(*)</code>              |
| <code>AVG()</code>      | Average value      | <code>AVG(Unit_Price)</code>       |
| <code>MIN()</code>      | Minimum value      | <code>MIN(Date)</code>             |
| <code>MAX()</code>      | Maximum value      | <code>MAX(Units_Sold)</code>       |
| <code>STDDEV()</code>   | Standard deviation | <code>STDDEV(Total_Revenue)</code> |
| <code>VARIANCE()</code> | Variance           | <code>VARIANCE(Units_Sold)</code>  |

#### SQL Example:

```

SELECT
    Product_Category,
    COUNT(*) as Transaction_Count,
    SUM(Total_Revenue) as Total_Revenue,
    AVG(Total_Revenue) as Average_Revenue,
    MIN(Total_Revenue) as Min_Revenue,
    MAX(Total_Revenue) as Max_Revenue,
    STDDEV(Total_Revenue) as Revenue_StdDev
FROM sales_data
GROUP BY Product_Category;

```

#### Important Rules:

- Must be used with `GROUP BY` when combining with non-aggregate columns

- Cannot be used in WHERE clause (use HAVING instead)
- Ignore NULL values (except COUNT(\*))
- Can be nested in some databases

## Question 5: How to handle NULLs in aggregates?

### Answer:

Aggregate functions handle NULLs differently, and there are several strategies to manage them effectively.

### Default NULL Behavior:

- SUM(), AVG(), MIN(), MAX() ignore NULLs
- COUNT(column) ignores NULLs
- COUNT(\*) includes rows with NULLs

### Handling Strategies:

#### 1. Replace NULLs with Default Values:

```
SELECT
    EXTRACT(MONTH FROM Date) as Month,
    SUM(COALESCE(Total_Revenue, 0)) as Monthly_Revenue,
    AVG(COALESCE(Total_Revenue, 0)) as Avg_Revenue
FROM sales_data
GROUP BY EXTRACT(MONTH FROM Date);
```

#### 2. Filter Out NULLs:

```
SELECT
    Product_Category,
    COUNT(*) as Total_Records,
    COUNT(Total_Revenue) as Non_NULL_Records,
    SUM(Total_Revenue) as Total_Revenue
FROM sales_data
WHERE Total_Revenue IS NOT NULL
GROUP BY Product_Category;
```

#### 3. Count NULLs Separately:

```
SELECT
    Product_Category,
    COUNT(*) as Total_Count,
    COUNT(Total_Revenue) as Valid_Revenue_Count,
    COUNT(*) - COUNT(Total_Revenue) as NULL_Count,
    SUM(Total_Revenue) as Total_Revenue
FROM sales_data
GROUP BY Product_Category;
```

#### 4. Conditional Aggregation:

```
SELECT
    EXTRACT(MONTH FROM Date) as Month,
    SUM(CASE WHEN Total_Revenue IS NOT NULL THEN Total_Revenue ELSE 0 END) as Revenue_Sum,
    COUNT(CASE WHEN Total_Revenue IS NOT NULL THEN 1 END) as Valid_Transactions
FROM sales_data
GROUP BY EXTRACT(MONTH FROM Date);
```

## Question 6: What's the role of ORDER BY and GROUP BY?

Answer:

| Clause   | Purpose                     | Execution Order         | Usage                     |
|----------|-----------------------------|-------------------------|---------------------------|
| GROUP BY | Groups rows for aggregation | 3rd (after WHERE)       | Required with aggregates  |
| ORDER BY | Sorts final result set      | Last (after everything) | Optional for presentation |

**Detailed Explanation:**

### GROUP BY:

- Combines rows with identical values in specified columns
- Creates groups for aggregate function calculations
- Must include all non-aggregate columns from SELECT
- Executed before aggregate calculations

### ORDER BY:

- Sorts the final result set after all processing
- Can sort by aggregated or non-aggregated columns
- Can use column aliases defined in SELECT
- Supports ASC (ascending) and DESC (descending)

**SQL Example:**

```
SELECT
    Product_Category,
    EXTRACT(MONTH FROM Date) as Month,
    SUM(Total_Revenue) as Monthly_Revenue,
    COUNT(*) as Transaction_Count
FROM sales_data
WHERE Date >= '2024-01-01'           -- 1. Filter rows first
GROUP BY Product_Category,          -- 2. Group filtered rows
    EXTRACT(MONTH FROM Date)
HAVING SUM(Total_Revenue) > 1000    -- 3. Filter groups
ORDER BY Product_Category,          -- 4. Sort final results
    Month DESC;
```

**SQL Query Execution Order:**

1. FROM - Select table

2. WHERE - Filter rows
3. GROUP BY - Group rows
4. HAVING - Filter groups
5. SELECT - Choose columns
6. ORDER BY - Sort results
7. LIMIT - Restrict output

## Question 7: How do you get the top 3 months by sales?

### Answer:

There are several methods to get top N results, each with different use cases.

#### Method 1: Using LIMIT (Simple)

```
SELECT
    EXTRACT(YEAR FROM Date) as Year,
    EXTRACT(MONTH FROM Date) as Month,
    SUM(Total_Revenue) as Monthly_Sales
FROM sales_data
GROUP BY EXTRACT(YEAR FROM Date), EXTRACT(MONTH FROM Date)
ORDER BY Monthly_Sales DESC
LIMIT 3;
```

#### Method 2: Using ROW\_NUMBER() Window Function

```
WITH monthly_sales AS (
    SELECT
        EXTRACT(YEAR FROM Date) as Year,
        EXTRACT(MONTH FROM Date) as Month,
        SUM(Total_Revenue) as Monthly_Sales,
        ROW_NUMBER() OVER (ORDER BY SUM(Total_Revenue) DESC) as sales_rank
    FROM sales_data
    GROUP BY EXTRACT(YEAR FROM Date), EXTRACT(MONTH FROM Date)
)
SELECT Year, Month, Monthly_Sales
FROM monthly_sales
WHERE sales_rank <= 3
ORDER BY sales_rank;
```

#### Method 3: Using RANK() for Ties

```
WITH ranked_months AS (
    SELECT
        EXTRACT(YEAR FROM Date) as Year,
        EXTRACT(MONTH FROM Date) as Month,
        SUM(Total_Revenue) as Monthly_Sales,
        RANK() OVER (ORDER BY SUM(Total_Revenue) DESC) as sales_rank
    FROM sales_data
    GROUP BY EXTRACT(YEAR FROM Date), EXTRACT(MONTH FROM Date)
)
```

```
SELECT Year, Month, Monthly_Sales, sales_rank
FROM ranked_months
WHERE sales_rank <= 3
ORDER BY sales_rank;
```

#### Method 4: Top N per Category

```
WITH category_monthly AS (
    SELECT
        Product_Category,
        EXTRACT(MONTH FROM Date) as Month,
        SUM(Total_Revenue) as Monthly_Sales,
        ROW_NUMBER() OVER (
            PARTITION BY Product_Category
            ORDER BY SUM(Total_Revenue) DESC
        ) as category_rank
    FROM sales_data
    GROUP BY Product_Category, EXTRACT(MONTH FROM Date)
)
SELECT Product_Category, Month, Monthly_Sales
FROM category_monthly
WHERE category_rank <= 3
ORDER BY Product_Category, category_rank;
```

#### When to Use Each Method:

- **LIMIT:** Fastest, simple top N
- **ROW\_NUMBER():** Exact N results, no ties
- **RANK():** Handles ties, may return more than N
- **DENSE\_RANK():** Consecutive ranking with ties

### Key Takeaways for Interview Success

#### Technical Skills to Emphasize:

1. **Date Functions:** Master EXTRACT(), DATE\_TRUNC(), DATEPART()
2. **Aggregation:** Understand all aggregate functions and their behaviors
3. **Window Functions:** Learn ROW\_NUMBER(), RANK(), LAG(), LEAD()
4. **NULL Handling:** Know COALESCE(), ISNULL(), IS NULL patterns
5. **Performance:** Understand indexing impact on GROUP BY queries

#### Common Pitfalls to Avoid:

1. Forgetting to include non-aggregate columns in GROUP BY
2. Using aggregate functions in WHERE clause instead of HAVING
3. Not considering NULL values in calculations

4. Mixing aggregate and non-aggregate columns incorrectly
5. Poor query performance due to missing indexes

**Best Practices:**

1. Always validate data quality before aggregation
2. Use meaningful aliases for calculated columns
3. Consider time zones in date grouping
4. Test queries with edge cases (NULLs, empty results)
5. Document complex queries for maintenance

**Generated on:** September 30, 2025

**Source:** Elevate Labs Data Analyst Internship - Task 6

**Topic:** Sales Trend Analysis Using Aggregations